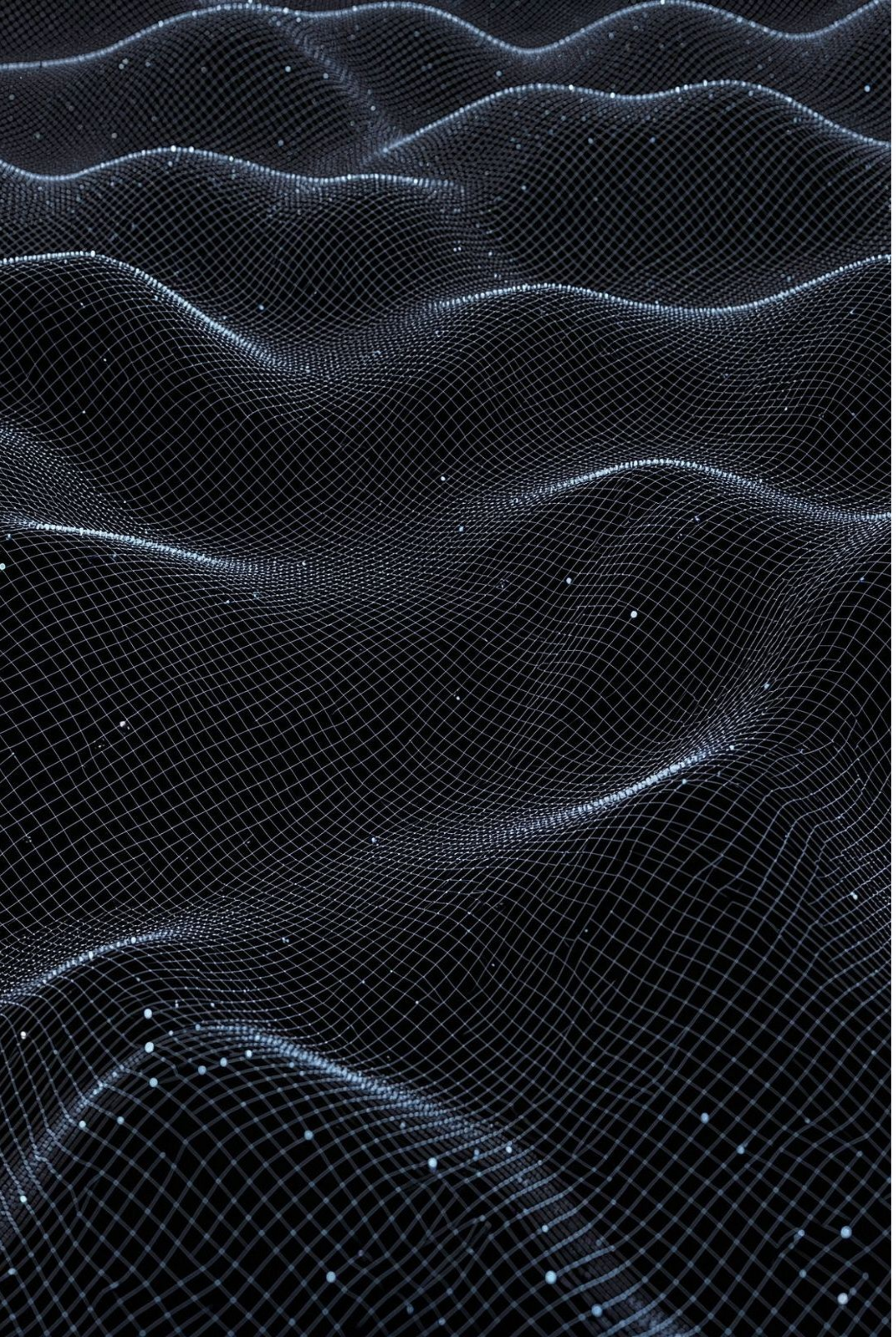
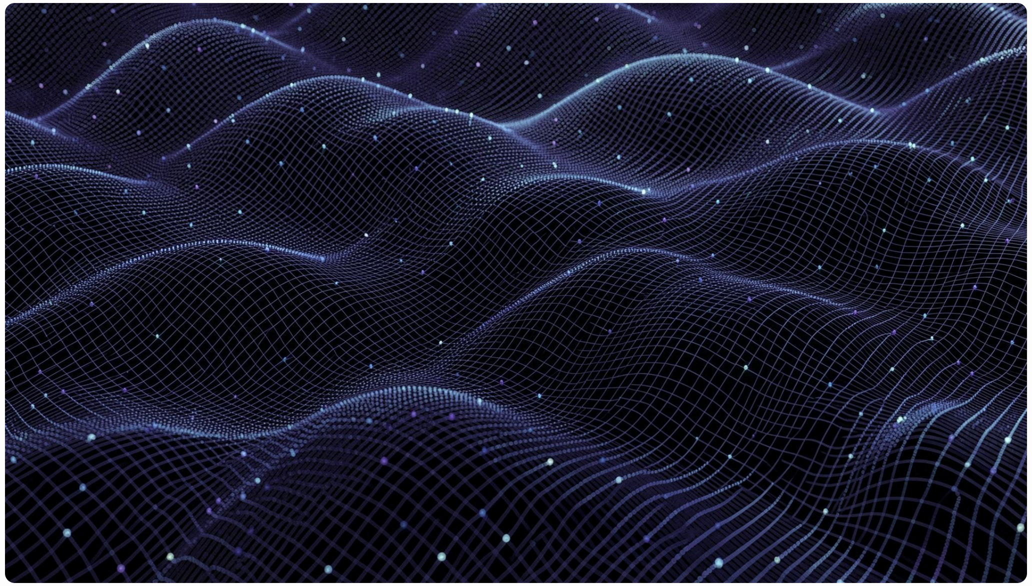


Deep Learning Fundamentals: From Research to Production

A comprehensive guide to PyTorch, Transformers, Distributed Training, Model Registry, CI/CD, and Docker — connecting every layer of the modern ML stack.

Presented by Priyanka

PyTorch & TensorFlow: Choosing Your Framework



PyTorch

Dynamic computation graphs (define-by-run) — modify network behavior on the fly. Debugging and experimentation feel natural. The dominant choice in research and academia.

TensorFlow

Production-grade ecosystem with mature tooling — TensorFlow Serving, TensorFlow Lite, and enterprise deployment support at scale.

Decision Rule

PyTorch for research & prototyping. **TensorFlow** for enterprise deployment pipelines requiring mature serving infrastructure.

Transformers: The Architecture Powering Modern AI

Self-Attention

Enables parallel processing of entire sequences — a fundamental leap over sequential RNNs.

Foundation Models

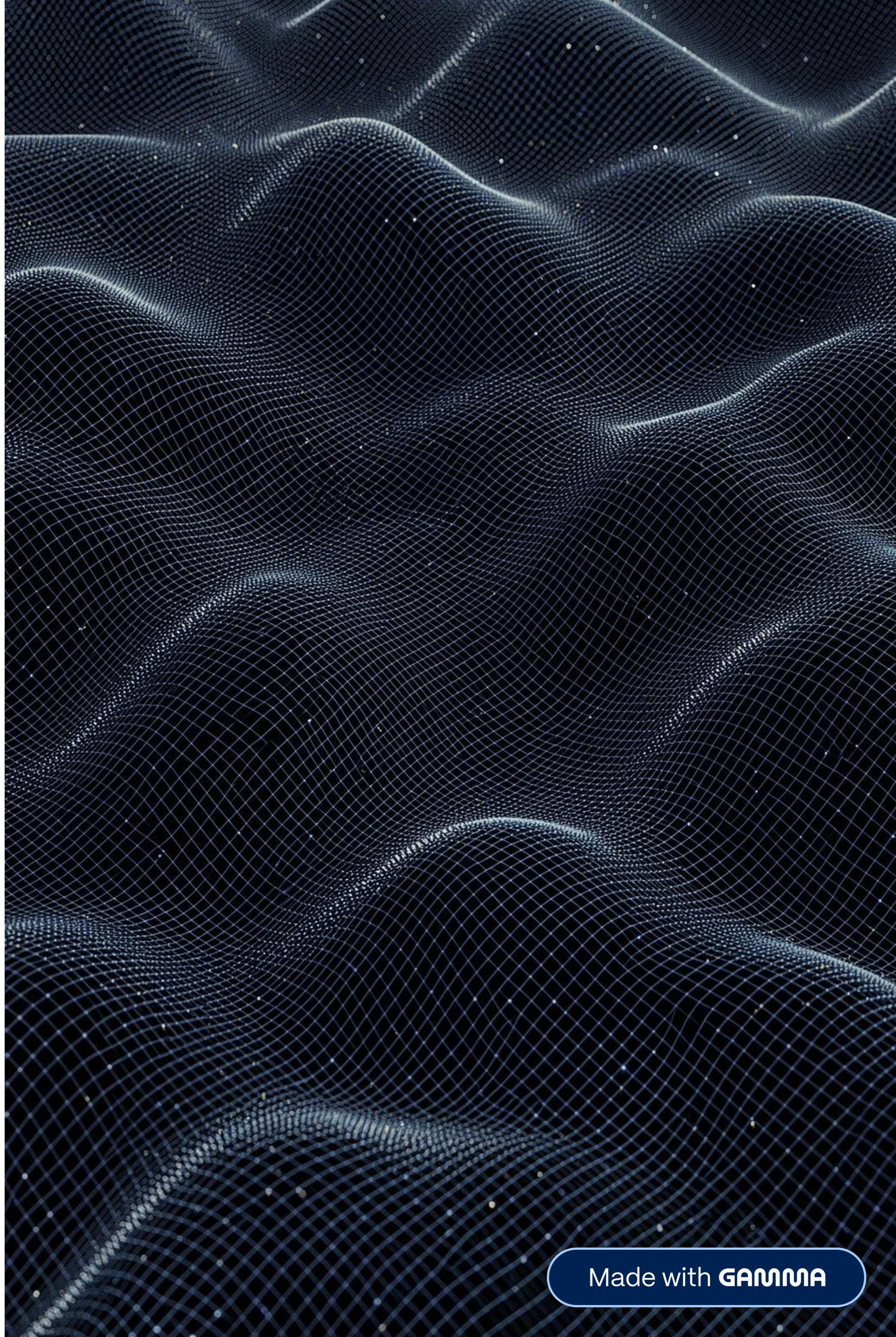
Powers GPT, BERT, T5, and every modern LLM. One architecture, unlimited applications.

Hugging Face

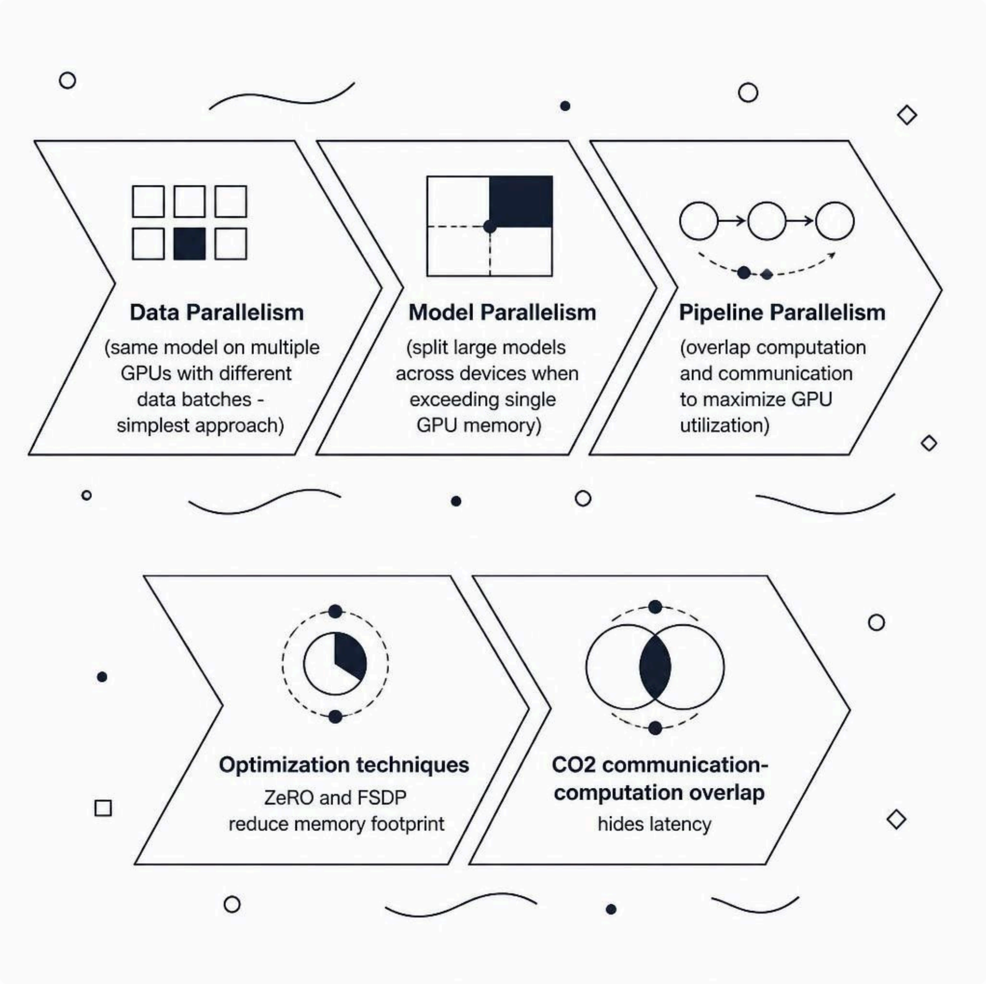
Pre-trained models for NLP, vision, and multimodal tasks — ready to use in minutes.

Transfer Learning

Fine-tune on your data in **hours, not weeks**. Pre-trained weights carry the heavy lifting.



Distributed Training: Scaling Across GPUs & TPUs



Why Distributed Training?

Modern LLMs exceed single-GPU memory by orders of magnitude. Distributed strategies unlock training at scale — but each comes with trade-offs in complexity and communication overhead.

- **Data Parallelism:** Batch data across GPUs, combine gradients. Simplest to implement.
- **Model Parallelism:** Slice the model itself across devices for memory relief.
- **Pipeline Parallelism:** Overlap compute and communication — modern libraries prevent GPU idle time using pipeline execution schedules.
- **ZeRO / FSDP:** Shard optimizer states and parameters to drastically cut memory footprint.

Model Registry: The Source of Truth for Production Models

Version & Track

Every model gets a unique ID, version, owner, and training commit hash. Immutable records prevent drift and enable rollback.

Essential Metadata

Model ID, dataset fingerprint, hyperparameters, evaluation metrics, and compliance tags — all stored together for full traceability.

Governance Gates

Automated validation, fairness checks, and manual approval workflows block any model that doesn't meet the bar before it touches production.

Lineage Tracking

Connect every model to its code commit, dataset version, and runtime environment. Reproducibility becomes a first-class guarantee.

CI/CD for ML: Automating the Full Pipeline

Why ML CI/CD Is Different

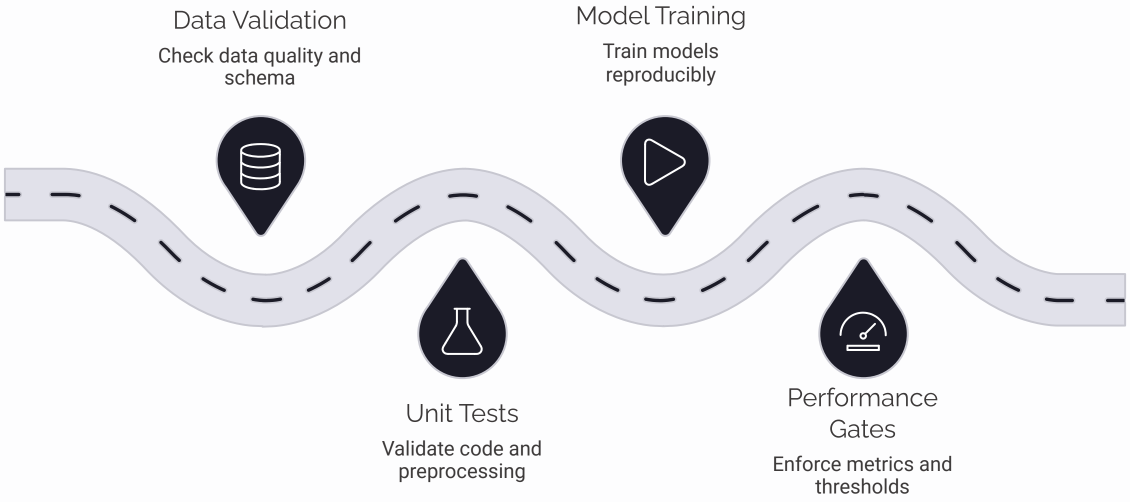
Traditional CI/CD tests code. **ML CI/CD tests code, data, and model quality together** — enabling reproducibility, automation, and separate dev/staging/production environments with scalable GPU/CPU resources.

Quality Gates

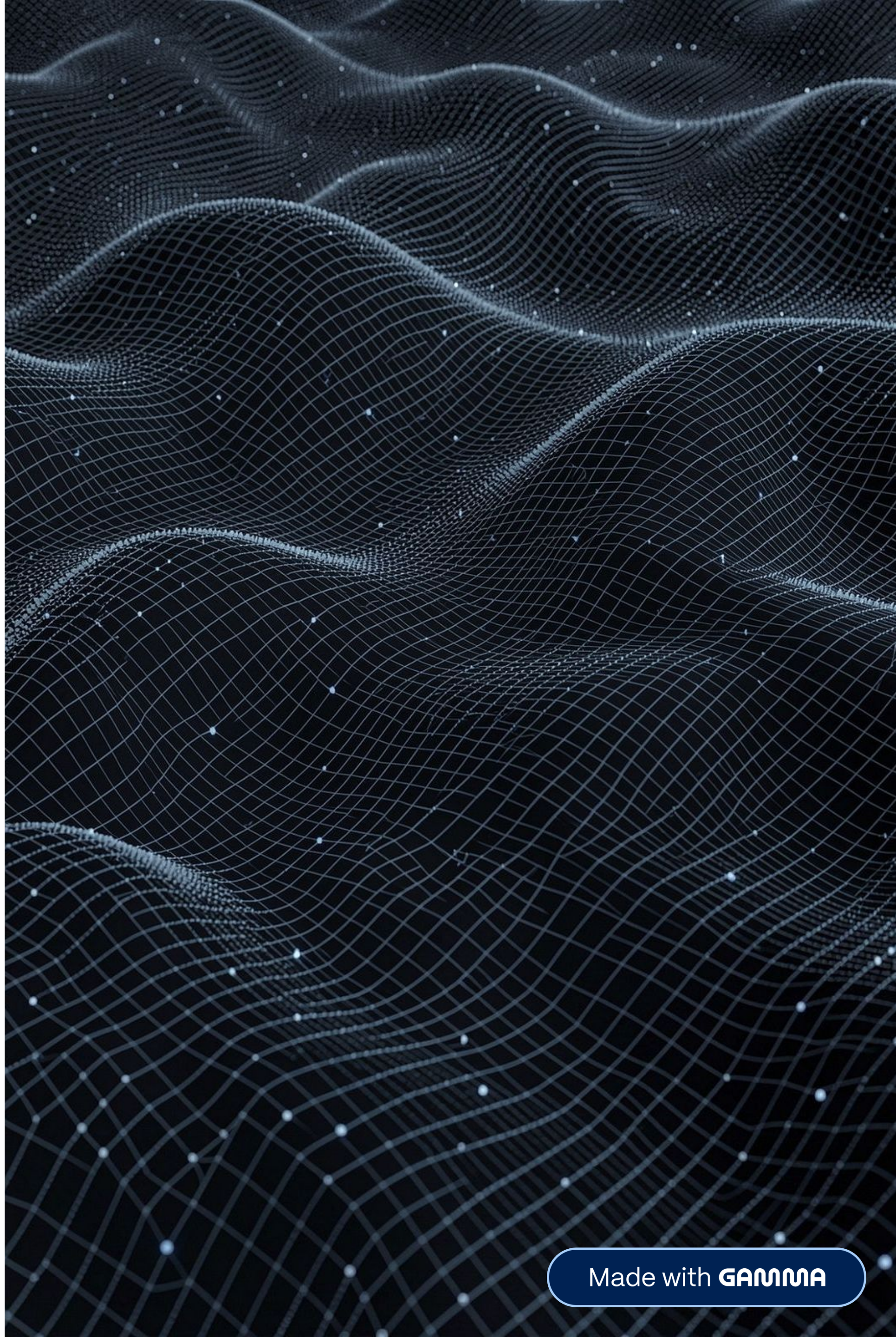
Minimum accuracy, F1, and AUC thresholds automatically block poor-performing models from advancing. No human bottleneck required.

Tools of the Trade

GitHub Actions, GitLab CI, and CML (Continuous Machine Learning) generate automated performance reports on every pull request.



Docker: Containerizing ML Workloads



1 Reproducible Environments

Code runs identically on your laptop, staging, and production. "Works on my machine" becomes a thing of the past.

2 ML Container Essentials

Base image: **nvidia/cuda** for GPU support. Add Python, PyTorch or TensorFlow, dependencies, and model artifacts in layers.

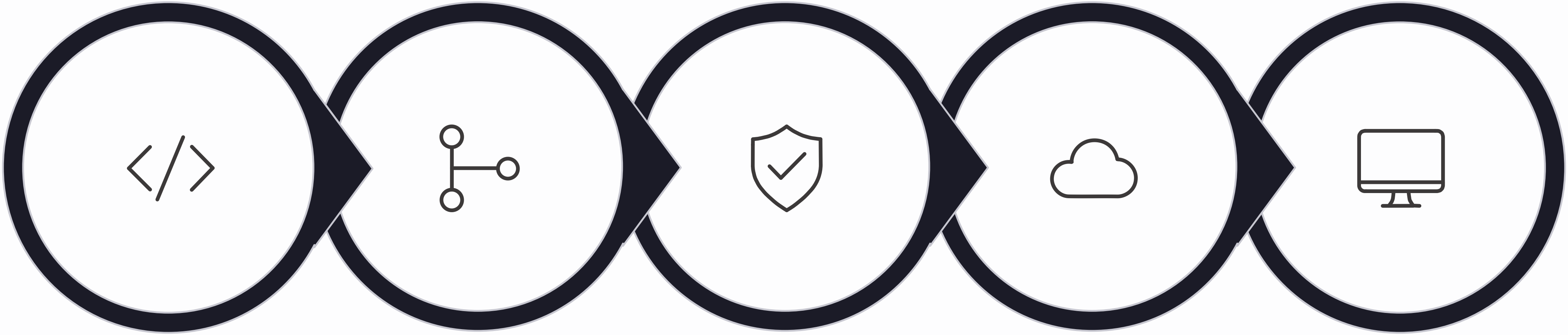
3 Multi-Stage Builds

Separate a large build stage from a lean runtime stage. Ship only what production needs — smaller, faster, more secure images.

4 Image Registries

Push to Docker Hub, Amazon ECR, or a private registry for versioned, deployment-ready images across every environment.

Putting It Together: End-to-End ML System



Development

Integration

Governance

Deployment

Monitoring

Every tool covered in this session connects into a single, automated lifecycle — from experiment to serving to monitoring. The goal: **reproducible, auditable, continuously improving ML systems**.

Thank You!